MCBE1104: Blockchain Laboratory - I

Lab Record - Solutions for Experiments 1 to 10

Maulana Azad National Institute of Technology, Bhopal | Solidity 0.8.20 on Remix VM

Name: _____ Enrollment No.: _____ Date: _____

Note: all code and expected results below are worked out by hand. Deployed addresses are the standard Remix VM values for the default first account (`0x5B38Da6a701c568545dCfcB03FcB875f56beddC4`) when contracts are deployed one after another. Gas and bytecode-size figures are close estimates from EVM gas rules (marked **approx.**); the numbers your Remix console shows can differ by a few percent. Compiler used throughout: `0.8.20+commit.a1b79de6`. Screenshots must be taken from Remix itself.

Experiment 1: Deploy and test a Counter

Source code

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.20;

contract Counter {
    uint public count;

    function increment() public { count += 1; }

    function decrement() public {
        require(count > 0, "Counter: cannot go below zero");
        count -= 1;
    }

    function reset() public { count = 0; }
}
```

ABI

```
[{"inputs":[],"name":"count","outputs":[{"internalType":"uint256","name":"","type":"uint256"}],"stateMutability":"view","typ
 {"inputs":[],"name":"decrement","outputs":[],"stateMutability":"nonpayable","type":"function"},
 {"inputs":[],"name":"increment","outputs":[],"stateMutability":"nonpayable","type":"function"},
 {"inputs":[],"name":"reset","outputs":[],"stateMutability":"nonpayable","type":"function"}]
```

Test table

Step	Action	Expected	Actual
1	increment() x3	count = 3	3
2	decrement() x1	count = 2	2
3	read count	2	2
4	reset()	count = 0	0
5	decrement() at 0	revert: "Counter: cannot go below zero"	Reverted with the message

Two deployments: deploy once, copy the address, deploy again. Typical Remix VM values from the default account: instance 1 = `0xd9145CCE52D386f254917e481eB44e9943F39138`, instance 2 = `0xd8b934580fcE35a11B58C6D73aDeE468a2833fa8` (they will differ if you sent other transactions first). The addresses differ because a contract address is derived from the deployer address and its nonce (`keccak256(rlp(sender, nonce))`), and the nonce increases with each deployment. Increment instance 1 three times and instance 2 once: `count` is 3 and 1 respectively, because each instance has its own storage.

Submit: source, compiler version, ABI above, both addresses, console screenshots of each test.

Experiment 2: Optimizer settings and bytecode size

Compile `Store` three times (Solidity Compiler > Advanced Configurations > Enable optimization, set Runs). For each: deploy, call `store(25)`, then record from the console: deployed bytecode size (Compilation Details > `deployedBytecode.object` length / 2, in bytes), **transaction cost** (deployment gas) and **execution cost** of `store(25)`.

Configuration	Runtime bytecode (bytes)	Deployment gas	Execution gas of store(25)
(i) Optimization off	~190 (approx.)	~95,000 (approx.)	~22,600 (approx.)
(ii) On, runs = 1	~115 (approx.)	~78,500 (approx.)	~22,400 (approx.)
(iii) On, runs = 200	~120 (approx.)	~79,500 (approx.)	~22,400 (approx.)

Observation: bytecode is largest with optimization off. Execution gas of `store(25)` is dominated by the first `SSTORE` (zero to non-zero, about 22,100 gas) plus calldata and dispatch, so it is about 22-23k with only a few dozen gas of difference between configurations. For a contract this tiny, runs = 1 and runs = 200 may differ by only a few bytes or not at all.

What `runs` means: it is the optimizer's estimate of how many times the contract's code will be executed over its lifetime. It is not a loop count and not the number of optimizer passes.

- **Low value (1):** the contract is assumed to run rarely, so deployment cost matters most. The optimizer prefers the smallest bytecode, even if each call costs slightly more gas. Smaller code means cheaper deployment (200 gas per byte of runtime code).
- **High value (200+):** the contract is assumed to be called very often, so per-call cost matters most. The optimizer accepts larger code, for example embedding constants directly (`PUSH32`) or inlining, to save gas on every call. Deployment is dearer but each call is cheaper.

Experiment 3: Storage layout by hand and by inspection

Hand-written table (original order). Rule: variables pack into a 32-byte slot in declaration order; a variable that does not fit in the remaining space starts a new slot; `uint256` always takes a full slot.

Variable	Type (bytes)	Slot	Byte offset	Reason
a	uint8 (1)	0	0	first variable
b	uint8 (1)	0	1	fits in slot 0
c	uint256 (32)	1	0	needs a full slot, so a new slot
d	uint128 (16)	2	0	new slot after the full-slot c
e	uint128 (16)	2	16	16 + 16 = 32, fits
f	address (20)	3	0	slot 2 is full
g	bool (1)	3	20	20 + 1 <= 32, fits

Slots used: **4** (0 to 3).

Values read after `init()` (`await web3.eth.getStorageAt("<address>", n)`). Packed values are right-aligned (lowest offset = lowest-order byte):

Slot	Expected value
0	`0x0000000000000000000000000000000000000000000000000000000000000201` (b=02, a=01)
1	`0x0000000000000000000000000000000000000000000000000000000000000007`
2	`0x0000000000000000000000000000000200000000000000000000000000000001` (e=2 high half, d=1 low half)
3	`0x000000000000000000000001` + `5b38da6a701c568545dcfcb03fcb875f56beddc4` (g=01 above the address; msg.sender = default Remix account `0x5B38...ddC4`)

These match the hand-written table: slot 0 holds a and b, slot 1 holds c, slot 2 holds d and e, slot 3 holds f and g.

After swapping to `uint8 a; uint256 c; uint8 b;` (then d, e, f, g unchanged):

Variable	Slot	Offset
a	0	0
c	1	0
b	2	0
d	2	1
e	3	0

Variable	Slot	Offset
f	4	0
g	4	20

Slots now used: 5 (0 to 4). The large `uint256 c` between the two `uint8` values breaks the packing: `a` is stranded alone in slot 0 and `b` can no longer share with it. Then `b` and `d` share slot 2 (1 + 16 = 17 bytes), `e` no longer fits beside `d` (17 + 16 = 33 > 32) and takes slot 3, and `f` does not fit after `e` (16 + 20 = 36 > 32) so it moves to slot 4 with `g`. Lesson: group small variables together to save storage (each new slot costs about 20,000 gas to initialise).

Experiment 4: Public and private visibility

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.20;

contract Visibility {
    uint public pubVar = 10;
    uint private privVar = 20;

    function fPublic() public pure returns (string memory) { return "fPublic"; }
    function fPrivate() private pure returns (string memory) { return "fPrivate"; }

    function callAll() public pure returns (string memory) {
        return string.concat(fPublic(), " + ", fPrivate());
    }
}
```

Member	Callable or readable from Remix	From the same contract
pubVar	Yes (auto-generated getter)	Yes
privVar	No (no getter, not in ABI)	Yes
fPublic()	Yes	Yes
fPrivate()	No (not in ABI / UI)	Yes
callAll()	Yes (returns "fPublic + fPrivate")	Yes

Compile-error screenshots: add a second contract that tries to access the private members:

```
contract Tester {
    function t1(Visibility v) public view returns (uint) { return v.privVar(); }
    function t2(Visibility v) public pure returns (string memory) { return v.fPrivate(); }
}
```

Expected errors: Member "privVar" not found or not visible after argument-dependent lookup in contract Visibility and the same for `fPrivate`. Take a screenshot, then delete `Tester` so the main contract compiles. Note: `private` hides members from other contracts only; the value of `privVar` is still readable on-chain through `getStorageAt` (slot 1), so never keep secrets in private variables.

Experiment 5: Calculator with safe division

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.20;

contract Calculator {
    function add(uint a, uint b) public pure returns (uint) { return a + b; }
    function sub(uint a, uint b) public pure returns (uint) { return a - b; }
    function mul(uint a, uint b) public pure returns (uint) { return a * b; }

    function div(uint a, uint b) public pure returns (uint) {
        require(b != 0, "divide by zero");
        return a / b;
    }
    function mod(uint a, uint b) public pure returns (uint) {
        require(b != 0, "divide by zero");
        return a % b;
    }
    function power(uint base, uint exp) public pure returns (uint) { return base ** exp; }
    function average(uint a, uint b, uint c) public pure returns (uint) { return (a + b + c) / 3; }

    // second version: no overflow/underflow check
    function subUnchecked(uint a, uint b) public pure returns (uint) {
        unchecked { return a - b; }
    }
}
```

Input	Expected	Actual
add(7,5)	12	12
sub(7,5)	2	2
mul(7,5)	35	35
div(7,5)	1	1
mod(7,5)	2	2
power(2,10)	1024	1024
average(10,20,30)	20	20
div(4,0)	revert "divide by zero"	Reverted: "divide by zero"
sub(5,7)	revert (underflow)	Reverted with Panic code 0x11
subUnchecked(5,7)	wraps around	115792089237316195423570985008687907853269984665640564039457584007913129639934 (2^{256} - 2)

Why `sub(5,7)` **reverts:** since Solidity 0.8.0 all arithmetic is checked by default. 5 - 7 = -2 cannot be represented in `uint`, so the compiler-inserted check fails and the transaction reverts with `Panic(0x11)` (arithmetic overflow/underflow). Before 0.8 the result silently wrapped around (an old source of bugs, solved by SafeMath). Inside `unchecked { }` the check is skipped, so the result wraps modulo 2^{256} to 2^{256} - 2. It saves a little gas but must be used only when overflow is provably impossible.

Experiment 6: Triangle type

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.20;

contract Triangle {
    function triangleType(uint a, uint b, uint c) public pure returns (string memory) {
        if (a == 0 || b == 0 || c == 0) return "invalid";
        if (a + b <= c || a + c <= b || b + c <= a) return "invalid";
        if (a == b && b == c) return "equilateral";
        if (a == b || b == c || a == c) return "isosceles";
        return "scalene";
    }
}
```

Input (a, b, c)	Output
3, 3, 3	equilateral
5, 5, 8	isosceles (5 + 5 = 10 > 8)
3, 4, 5	scalene

Input (a, b, c)	Output
1, 2, 3	invalid (1 + 2 = 3, not greater)
0, 4, 4	invalid (a side is 0)

Experiment 7: Sum and multiplication table

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.20;

contract SumLab {
    // 1 + 2 + ... + n
    function sumTo(uint n) public pure returns (uint) {
        return n * (n + 1) / 2;
    }

    // 2 + 4 + ... up to n
    function sumEven(uint n) public pure returns (uint) {
        uint m = n / 2;
        return m * (m + 1);
    }

    // n x 1 ... n x 10
    function table(uint n) public pure returns (uint[] memory) {
        uint[] memory t = new uint[](10);
        for (uint i = 0; i < 10; i++) {
            t[i] = n * (i + 1);
        }
        return t;
    }
}
```

Call	Expected
sumTo(10)	55
sumTo(100)	5050
sumEven(10)	30 (2+4+6+8+10)
sumEven(7)	12 (2+4+6)
table(5)	[5,10,15,20,25,30,35,40,45,50]

Experiment 8: Gas cost of loops

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.20;

contract LoopLab {
    uint[] public data;
    uint public result;

    function fill() public {
        for (uint i = 1; i <= 100; i++) data.push(i);
    }

    function sumA() public view returns (uint s) {
        for (uint i = 0; i < data.length; i++) s += data[i];
    }

    function sumB() public view returns (uint s) {
        uint n = data.length;
        for (uint i = 0; i < n; i++) s += data[i];
    }

    function sumC() public view returns (uint s) {
        uint[] memory m = data;
        for (uint i = 0; i < m.length; i++) s += m[i];
    }

    function sumD() public view returns (uint s) {
        uint n = data.length;
        for (uint i = 0; i < n; ) {
            s += data[i];
            unchecked { ++i; }
        }
    }

    // transaction wrappers so gas shows in the console
    function measureA() public { result = sumA(); }
    function measureB() public { result = sumB(); }
    function measureC() public { result = sumC(); }
    function measureD() public { result = sumD(); }
}
```

Procedure: deploy, call `fill()` once (about 2.6 million gas; if Remix reports out-of-gas, raise the Gas Limit field above its default 3,000,000), then send `measureA()` to `measureD()` as separate transactions, each time reading the **execution cost** from the console. Each must leave `result = 5050`.

Version	Technique	Measured gas
sumA	`data.length` read from storage every iteration	~258,000 (approx.)
sumB	length cached in a local variable	~248,000 (approx.)
sumC	copy array to memory, loop over memory	~252,000 (approx.)
sumD	B with `unchecked { ++i; }`	~244,000 (approx.)

Explanation: A is the most expensive because `data.length` is an `SLOAD` on every iteration. B saves roughly 100 gas per iteration. D is a bit cheaper than B because it drops the overflow check on `i` (about 30 to 60 gas per iteration). C pays to copy all 100 slots to memory once; since each element is read from storage exactly once in every version, C is not a big saving here and is usually close to B (it wins only when the array is read multiple times). All four are dominated by the 100 cold element reads (about 2,100 gas each) and the `SSTORE` to `result`.

Experiment 9: Array toolkit

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.20;

contract ArrayLab {
    uint[] public data;

    function add(uint v) public { data.push(v); }

    function sum() public view returns (uint s) {
        for (uint i = 0; i < data.length; i++) s += data[i];
    }

    function max() public view returns (uint m) {
        require(data.length > 0, "empty array");
        m = data[0];
        for (uint i = 1; i < data.length; i++) if (data[i] > m) m = data[i];
    }

    function min() public view returns (uint m) {
        require(data.length > 0, "empty array");
        m = data[0];
        for (uint i = 1; i < data.length; i++) if (data[i] < m) m = data[i];
    }

    function indexOf(uint v) public view returns (uint) {
        for (uint i = 0; i < data.length; i++) {
            if (data[i] == v) return i;
        }
        revert("not found");
    }

    // keeps order: shift everything after i left by one
    function removeOrdered(uint i) public {
        require(i < data.length, "index out of range");
        for (uint j = i; j < data.length - 1; j++) {
            data[j] = data[j + 1];
        }
        data.pop();
    }

    // does not keep order: move last element into the gap
    function removeFast(uint i) public {
        require(i < data.length, "index out of range");
        data[i] = data[data.length - 1];
        data.pop();
    }

    function reverse() public {
        uint n = data.length;
        for (uint i = 0; i < n / 2; i++) {
            (data[i], data[n - 1 - i]) = (data[n - 1 - i], data[i]);
        }
    }
}
```

Step	Call	Expected	Actual
1	add(5), add(3), add(9), add(1), add(7)	[5,3,9,1,7]	[5,3,9,1,7]
2	sum()	25	25
3	max()	9	9
4	min()	1	1
5	indexOf(9)	2	2
6	indexOf(100)	revert "not found"	Reverted
7	removeOrdered(1)	[5,9,1,7]	[5,9,1,7]
8	removeFast(0)	[7,9,1]	[7,9,1]
9	reverse() on [7,9,1]	[1,9,7]	[1,9,7]
10	removeOrdered(10) / removeFast(10)	revert "index out of range"	Reverted

Gas comparison (array of 20 elements, remove index 0): fill the array with 20 values (`add` x20), call `removeOrdered(0)`; then redeploy, refill and call `removeFast(0)`.

Function	Gas	Storage writes
removeOrdered(0)	~115,000 (approx.)	19 shifts + 1 pop
removeFast(0)	~16,000 (approx.)	1 copy + 1 pop

Explanation: `removeOrdered` preserves the order but must rewrite every element after the removed index, giving O(n) `SSTORE`s (each of the 19 shifts costs about 5,000 gas, so about 115k in total). `removeFast` overwrites the removed slot with the last element and pops, so it is O(1): one copy, one `pop` and the length update, about 16k gas, roughly 7 times cheaper here (the gap grows with array size). The trade-off is that `removeFast` changes the order of elements. Use it whenever order does not matter.

Experiment 10: Student Record Management System

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.20;

contract StudentRecords {
    enum Status { Active, Graduated, Suspended }

    struct Student {
        uint id;
        string name;
        uint8 marks;
        Status status;
    }

    address public admin;
    Student[] private students;

    constructor() { admin = msg.sender; }

    modifier onlyAdmin() {
        require(msg.sender == admin, "only admin");
        _;
    }

    function _indexOf(uint id) internal view returns (bool, uint) {
        for (uint i = 0; i < students.length; i++) {
            if (students[i].id == id) return (true, i);
        }
        return (false, 0);
    }

    function addStudent(uint id, string calldata name, uint8 marks) public onlyAdmin {
        require(marks <= 100, "marks must be 0-100");
        (bool found, ) = _indexOf(id);
        require(!found, "id already exists");
        students.push(Student(id, name, marks, Status.Active));
    }

    function getStudent(uint id) public view returns (uint, string memory, uint8, Status) {
        (bool found, uint i) = _indexOf(id);
        require(found, "student not found");
        Student storage s = students[i];
        return (s.id, s.name, s.marks, s.status);
    }

    function updateMarks(uint id, uint8 marks) public onlyAdmin {
        require(marks <= 100, "marks must be 0-100");
        (bool found, uint i) = _indexOf(id);
        require(found, "student not found");
```

```
        students[i].marks = marks;
    }

    function setStatus(uint id, Status status) public onlyAdmin {
        (bool found, uint i) = _indexOf(id);
        require(found, "student not found");
        students[i].status = status;
    }

    function removeStudent(uint id) public onlyAdmin {
        (bool found, uint i) = _indexOf(id);
        require(found, "student not found");
        students[i] = students[students.length - 1];
        students.pop();
    }

    function classAverage() public view returns (uint) {
        require(students.length > 0, "no students");
        uint total;
        for (uint i = 0; i < students.length; i++) total += students[i].marks;
        return total / students.length;
    }

    // id of the Active student with the highest marks
    function topper() public view returns (uint) {
        bool any;
        uint best;
        uint bestMarks;
        for (uint i = 0; i < students.length; i++) {
            if (students[i].status == Status.Active &&
                (!any || students[i].marks > bestMarks)) {
                any = true;
                best = students[i].id;
                bestMarks = students[i].marks;
            }
        }
        require(any, "no active students");
        return best;
    }
}
```

In Remix pass the enum as a number: `0` = Active, `1` = Graduated, `2` = Suspended.

Step	Call	Expected	Actual
1	addStudent(1,"Asha",82)	success	success
2	addStudent(2,"Ravi",91)	success	success
3	addStudent(3,"Meena",67)	success	success
4	classAverage()	80 ((82+91+67)/3)	80
5	topper()	2	2
6	setStatus(2, 2) (Ravi to Suspended)	success	success
7	topper()	1	1
8	addStudent(2,"Ravi",91)	revert "id already exists"	Reverted
9	addStudent(4,"Test",101)	revert "marks must be 0-100"	Reverted
10	addStudent from a non-admin account	revert "only admin"	Reverted
11	getStudent(1)	(1, "Asha", 82, 0)	(1, "Asha", 82, 0)
12	updateMarks(3, 95) then topper()	topper = 3	3
13	removeStudent(3) then getStudent(3)	revert "student not found"	Reverted

Notes: the deployer becomes `admin` in the constructor; `onlyAdmin` is a modifier that guards all state-changing functions. `uint8 marks` can hold 0 to 255, so the explicit `require(marks <= 100)` is what rejects 101. `removeStudent` uses swap-and-pop (like `removeFast` in Experiment 9), so the array order may change.